CLIENT_STATE_MACHINE

Helix OTA — Client State Machine

Overview

This state diagram models the **complete lifecycle of an update on the device side**. The Helix OTA Client (C++ daemon on Android) transitions through well-defined states as it checks for, downloads, verifies, installs, and commits an update. Every transition is reported to the server, and error paths lead to well-defined recovery behavior.

Diagram

```
stateDiagram-v2
    direction TB

    [*] --> IDLE : Client daemon started

    IDLE --> CHECKING : Timer fires<br/>(default: every 4h)<br/>OR Operator forces check

    CHECKING --> IDLE : No update available<br/>(server responds 204)
    CHECKING --> UPDATE_AVAILABLE : Update found<br/>(server responds 200 + metadata)
    CHECKING --> IDLE : Network error<br/>(retry on next timer)

    UPDATE_AVAILABLE --> DOWNLOADING : Auto-download policy<br/>OR User approves<br/>(for metered connections)
    UPDATE_AVAILABLE --> IDLE : User defers<br/>(max 3 deferrals,<br/>then forced)

    DOWNLOADING --> DOWNLOADING : Progress update<br/>(stream chunks)
    DOWNLOADING --> VERIFYING : Download 100% complete<br/>+ file size matches
    DOWNLOADING --> IDLE : Network timeout<br/>(retry with resume<br/>at next check-in)
    DOWNLOADING --> FAILED : Disk full<br/>OR Storage write error
```

```
    VERIFYING --> INSTALLING : SHA-256 hash ✓<br/>AND Ed25519
signature ✓
    VERIFYING --> DOWNLOADING : Hash mismatch<br/>(retry
download,<br/>max 3 attempts)
    VERIFYING --> FAILED : Signature invalid<br/>(non-
recoverable,<br/>possible tampering)

    INSTALLING --> INSTALLING : update_engine<br/>progress updates
    INSTALLING --> REBOOTING : update_engine reports<br/
>ApplyComplete(success)
    INSTALLING --> FAILED : update_engine reports<br/
>ApplyComplete(error)<br/>OR Process killed

    REBOOTING --> COMMITTING : Device rebooted<br/>successfully
into<br/>new slot
    REBOOTING --> FAILED : Boot verification failed<br/>(A/B auto-
rollback<br/>to old slot)

    COMMITTING --> SUCCEEDED : update_engine<br/
>MarkSlotSuccessful() ✓
    COMMITTING --> FAILED : MarkSlotSuccessful<br/>failed<br/
>(device on old slot)

    FAILED --> IDLE : Report sent to server<br/>+ Error logged
locally<br/>+ Backoff before retry

    SUCCEEDED --> IDLE : Report sent to server<br/>+ Update
complete<br/>+ Resume normal check cycle

    state IDLE {
        direction LR
        [*] --> ScheduledCheck
        ScheduledCheck : Waiting for next<br/>check timer
        ScheduledCheck --> ForcedCheck : Operator triggers<br/
>immediate check
    }

    state DOWNLOADING {
        direction LR
        [*] --> ResolvingURL
        ResolvingURL --> Streaming : Presigned URL<br/>obtained
        Streaming --> Streaming : Chunk received,<br/>written to
disk
        Streaming --> Resuming : Connection lost<br/>(partial
download)
        Resuming --> Streaming : Connection restored<br/>(Range
header)
    }

    state VERIFYING {
        direction LR
        [*] --> HashCheck
        HashCheck : SHA-256 comparison<br/>of entire file
```

```
        HashCheck --> SignatureCheck : Hash ✓
        HashCheck : Signature uses Ed25519<br/>with embedded<br/
>public key
        SignatureCheck --> Verified : Signature ✓
    }

    state INSTALLING {
        direction LR
        [*] --> ApplyingPayload
        ApplyingPayload : update_engine writes<br/>to inactive A/B
slot
        ApplyingPayload --> PostInstall : Payload applied
        PostInstall : Running post-install<br/>scripts (if any)
    }

    note right of IDLE
        Default check interval: 4 hours
        Jitter: ±30 min to spread
        server load. Operator can
        force immediate check.
    end note

    note right of UPDATE_AVAILABLE
        Server provides:
        - artifact_url
        - target_build
        - hash_sha256
        - ed25519_signature
        - size_bytes
        - metadata (changelog, urgency)
    end note

    note right of FAILED
        Error codes:
        - NETWORK_ERROR
        - HASH_MISMATCH
        - INVALID_SIGNATURE
        - DISK_FULL
        - INSTALL_ERROR
        - BOOT_FAILURE
        - COMMIT_FAILURE
        All reported to server
        with diagnostic details.
    end note

    note right of SUCCEEDED
        Final state reports:
        - new_build fingerprint
        - install_duration
        - total_download_size
        Device returns to IDLE
        and resumes normal
        check cycle.
    end note
```

State Descriptions

State	Description	Duration (typical)	Server Report
IDLE	Waiting for next scheduled check or forced check	0–4 hours	—
CHECKING	Sending device profile to server, awaiting response	1–5 seconds	—
UPDATE_AVAILABLE	Server returned update metadata; awaiting download approval	0–24 hours (defer)	—
DOWNLOADING	Streaming artifact from MinIO via presigned URL	1–30 minutes	Progress % every 10%
VERIFYING	Computing SHA-256 hash and verifying Ed25519 signature	10–60 seconds	State transition only
INSTALLING	update_engine writing payload to inactive A/B slot	2–15 minutes	Progress % every 10%
REBOOTING	Device rebooting into new slot	30–90 seconds	State transition only
COMMITTING	Marking new slot as successful (prevents rollback)	< 5 seconds	State transition only
SUCCEEDED	Update fully committed and reported to server	—	Final success report
FAILED	Update failed at any stage; error reported	—	Error code + details

Retry Policy

Error Type	Max Retries	Backoff Strategy	Notes
Network timeout	5		

Error Type	Max Retries	Backoff Strategy	Notes
		Exponential: 1m, 2m, 4m, 8m, 16m	Resumes partial download
Hash mismatch	3	Linear: immediate retry	Full re-download required
Invalid signature	0	No retry	Security halt — alert operator
Disk full	0	No retry	User intervention needed
Install error	1	30 min delay	A/B slot automatically recovered
Boot failure	0	No retry	A/B auto-rollback; report to server
Commit failure	1	Immediate retry	Rare; usually transient

Deferral Policy

- **Metered connections**: User may defer update up to **3 times**
- **Critical/Security updates**: Deferral limited to **7 days** maximum
- **After 3 deferrals**: Update is force-downloaded on next check (regardless of network type)